

1-

```
postgres=# CREATE TABLE track (  
    track_id SERIAL PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    description TEXT  
);  
CREATE TABLE  
postgres=# CREATE TABLE subject (  
    sub_id SERIAL PRIMARY KEY,  
    sub_name VARCHAR(100) NOT NULL,  
    max_score INTEGER  
);  
CREATE TABLE  
postgres=# CREATE TABLE exam (  
    exam_id SERIAL PRIMARY KEY,  
    date DATE NOT NULL  
);  
CREATE TABLE
```

```
CREATE TABLE  
postgres=# CREATE TABLE grades (  
    stu_id INTEGER NOT NULL,  
    sub_id INTEGER NOT NULL,  
    exam_id INTEGER NOT NULL,  
    grade VARCHAR(2) NOT NULL,  
    PRIMARY KEY (stu_id, sub_id, exam_id),  
    CONSTRAINT fk_grades_student  
        FOREIGN KEY (stu_id) REFERENCES student(stu_id)  
        ON DELETE CASCADE,  
    CONSTRAINT fk_grades_subject  
        FOREIGN KEY (sub_id) REFERENCES subject(sub_id)  
        ON DELETE CASCADE,  
    CONSTRAINT fk_grades_exam  
        FOREIGN KEY (exam_id) REFERENCES exam(exam_id)  
        ON DELETE CASCADE  
);
```

```
postgres=#  
CREATE TABLE student(  
    stu_id SERIAL PRIMARY KEY,  
    e_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    address TEXT,  
    track_id INTEGER,  
    CONSTRAINT fk_track  
        FOREIGN KEY (track_id)  
        REFERENCES track(track_id)  
);  
CREATE TABLE
```

```

postgres=# CREATE TABLE stu_sub (
    stu_id INTEGER NOT NULL,
    sub_id INTEGER NOT NULL,
    enrollment_date DATE DEFAULT CURRENT_DATE,
    PRIMARY KEY (stu_id, sub_id),
    CONSTRAINT fk_stusub_student
        FOREIGN KEY (stu_id) REFERENCES student(stu_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_stusub_subject
        FOREIGN KEY (sub_id) REFERENCES subject(sub_id)
        ON DELETE CASCADE
);

```

```

postgres=# CREATE TABLE track_sub (
    track_id INTEGER NOT NULL,
    sub_id INTEGER NOT NULL,
    is_core BOOLEAN DEFAULT TRUE,
    semester INTEGER,
    PRIMARY KEY (track_id, sub_id),
    CONSTRAINT fk_tracksub_track
        FOREIGN KEY (track_id) REFERENCES track(track_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_tracksub_subject
        FOREIGN KEY (sub_id) REFERENCES subject(sub_id)
        ON DELETE CASCADE
);

```

2-

```

postgres=# INSERT INTO subject (sub_name, max_score) VALUES
('Database Systems', 100),
('Calculus I', 100),
('Data Structures', 100);
INSERT 0 3
postgres=# INSERT INTO student (e_name, email, address, track_id) VALUES
('aya', 'aya@gmail.com', 'feryal', 1),
('menna', 'menna@gmail.com', 'city', 3),
('arwa', 'arwa@gmail.com', 'olta', 1);
INSERT 0 3
postgres=# INSERT INTO exam (date) VALUES
('2023-09-15'),
('2023-12-10'),
('2024-03-20');
INSERT 0 3
postgres=# INSERT INTO track_sub (track_id, sub_id, is_core, semester) VALUES
(1, 1, TRUE, 3), -- CS requires Databases in semester 3
(1, 2, TRUE, 1), -- CS requires Calculus in semester 1
(2, 3, TRUE, 1), -- Business requires Accounting in semester 1
(3, 2, TRUE, 1), -- Engineering requires Calculus

```

3-

```
postgres=# ALTER TABLE student
postgres=#
ADD COLUMN birth_date DATE;
ALTER TABLE
postgres=# UPDATE student SET birth_date = '2000-05-15' WHERE stu_id = 1;
UPDATE student SET birth_date = '2001-08-22' WHERE stu_id = 2;
UPDATE student SET birth_date = '1999-11-03' WHERE stu_id = 3;
UPDATE 1
UPDATE 1
UPDATE 1
postgres=#
```

4-

```
postgres=# ALTER TABLE student
ADD COLUMN gender gender_enum;
ALTER TABLE
postgres=# UPDATE student SET gender = 'Female' WHERE stu_id IN (1, 4)
UPDATE student SET gender = 'Male' WHERE stu_id IN (2, 3);
ERROR:  syntax error at or near "UPDATE"
LINE 2: UPDATE student SET gender = 'Male' WHERE stu_id IN (2, 3);
        ^
postgres=# UPDATE student SET gender = 'Female' WHERE stu_id IN (1, 4);
UPDATE student SET gender = 'Male' WHERE stu_id IN (2, 3);
UPDATE 1
UPDATE 2
```

6-

```
postgres=# SELECT
    stu_id,
    e_name AS student_name,
    birth_date,
    EXTRACT(YEAR FROM AGE(birth_date)) AS age
FROM
    student
WHERE
    gender = 'Male'
    AND birth_date < '1991-10-01'
ORDER BY
    birth_date;
 stu_id | student_name | birth_date | age
-----+-----+-----+-----
(0 rows)

postgres=#
```

7-

```
postgres=# SELECT
    stu_id,
    e_name AS student_name,
    email,
    birth_date
FROM
    student
WHERE
    e_name LIKE 'a%'
ORDER BY
    e_name;
 stu_id | student_name |      email       | birth_date
-----+-----+-----+-----
      3 | arwa         | arwa@gmail.com   | 1999-11-03
      1 | aya          | aya@gmail.com    | 2000-05-15
(2 rows)
```

8-

```
postgres=# SELECT
    sub_id AS "Subject ID",
    sub_name AS "Subject Name",
    max_score AS "Maximum Score"
FROM
    subject
ORDER BY
    max_score DESC;
 Subject ID | Subject Name | Maximum Score
-----+-----+-----
          1 | Database Systems |          100
          2 | Calculus I      |          100
          3 | Data Structures |          100
(3 rows)

postgres=#
```

9-

```
postgres=# SELECT
    sub_id AS "Subject ID",
    sub_name AS "Subject Name",
    max_score AS "Maximum Score"
FROM
    subject
WHERE
    max_score = (SELECT MAX(max_score) FROM subject)
ORDER BY
    sub_name;
 Subject ID | Subject Name | Maximum Score
-----+-----+-----
          2 | Calculus I   |           100
          1 | Database Systems |           100
          3 | Data Structures |           100
(3 rows)
```

Lab2

1-

```
postgres=# SELECT
    COUNT(*) AS number_of_mohammeds
FROM
    student
WHERE
    e_name = 'Mohammed';
 number_of_mohammeds
-----
                   0
(1 row)

postgres=#
```

2-

```
postgres=# SELECT
  gender,
  COUNT(*) AS student_count
FROM
  student
WHERE
  gender IN ('Male', 'Female')
GROUP BY
  gender
ORDER BY
  gender;
 gender | student_count 
-----+-----
 Male   |              2
 Female |              1
(2 rows)
```

```
postgres=#
```

3-

```
postgres=# SELECT
  SPLIT_PART(e_name, ' ', 1) AS first_name,
  COUNT(*) AS name_count
FROM
  student
GROUP BY
  SPLIT_PART(e_name, ' ', 1)
HAVING
  COUNT(*) > 2
ORDER BY
  name_count DESC;
 first_name | name_count 
-----+-----
(0 rows)
```

```
postgres=#
```

4-

```
postgres=# SELECT
    s.stu_id AS "Student ID",
    s.e_name AS "Student Name",
    t.name AS "Track Name",
    t.description AS "Track Description"
FROM
    student s
LEFT JOIN
    track t ON s.track_id = t.track_id
ORDER BY
    s.e_name;
```

Student ID	Student Name	Track Name	Track Description
3	arwa	CS	Focus on programming and algorithms
1	aya	CS	Focus on programming and algorithms
2	menna	EE	Circuit design and electronics

(3 rows)

5-

```
postgres=# SELECT
    s.stu_id AS "Student ID",
    s.e_name AS "Student Name",
    t.name AS "Track Name"
FROM
    student s
LEFT JOIN
    track t ON s.track_id = t.track_id
WHERE
    t.name != 'OS' OR t.name IS NULL
ORDER BY
    s.e_name;
```

Student ID	Student Name	Track Name
3	arwa	CS
1	aya	CS
2	menna	EE

(3 rows)